

WAF -to-SIEM — Part 2:

Closing the Loop: WAF to SIEM Log Integration and Live Attack Validation

I forwarded a containerized Web Application Firewall's logs into a cloud SIEM, resolved a broken audit log pipeline along the way, then validated detection end to end with real cross VM attacks.

Overview

Part 2 of this project identified a documented detection gap: a network IDS signature (Suricata) failed to reliably fire on a successful SQL injection attack against the Damn Vulnerable Web Application (DVWA). In this phase, I closed that gap from a different angle, adding a Web Application Firewall (WAF) directly in front of the vulnerable application, then integrating its logs into the SIEM so that WAF decisions are visible and searchable alongside every other data source in the lab.

The result is a defense in depth pipeline: a containerized ModSecurity WAF now sits in front of DVWA and blocks malicious requests inline, its logs are captured and forwarded into the Wazuh SIEM, and the SIEM's own ruleset independently classifies the traffic as a SQL injection attempt, producing two correlated, MITRE mapped alerts per attack. I validated the pipeline with real attacks launched from the attacker VM, not synthetic test data.

Lab Architecture

My environment is unchanged from prior phases, with one new component added in front of the victim's web application:

Component	Role
ManKali-attacker	Kali Linux VM, attacker machine, source of all offensive traffic
ubuntu-victim	Ubuntu Server VM, hosts DVWA (Docker), the new ModSecurity WAF (Docker), Suricata IDS, and the Wazuh agent
modsec-waf (new)	Docker container running OWASP ModSecurity + CRS as an nginx reverse proxy in front of DVWA, listening on port 8081
wazuh-manager	Cloud hosted Wazuh SIEM stack (DigitalOcean VPS), receives and correlates events from the agent

Network addressing is VirtualBox's default NAT Network range: the attacker sits at 10.0.2.15 and the victim at 10.0.2.3. The WAF proxies port 8081 to DVWA's backend on port 8080, so all traffic intended to pass through ModSecurity must target 8081 specifically.

The Problem: A Broken Audit Log Pipeline

ModSecurity's own audit logging was configured to write JSON output only to the container's stdout, with no persistent file on the host for Wazuh to read. I first attempted redirecting the audit log to a file via environment variables and volume mounts, including correcting a host directory ownership mismatch (the log directory needed to be owned by UID 101, matching the container's nginx user), but the target file remained at 0 bytes despite every configuration path pointing correctly to it. I documented this as an open item rather than masking it.

The workaround

Rather than continue chasing the native audit log bug, I wrote a small systemd service to capture the container's logs directly and persist them to a host file:

```
ExecStart=/bin/bash -c "/usr/bin/docker logs -f modsec-waf >> /var/log/modsec-waf.log 2>&1"
```

I enabled this service (`modsec-log-forward.service`) and confirmed it running, providing a stable, persistent log source independent of whatever was blocking ModSecurity's native audit output, a pragmatic fix that unblocked the integration without needing the root cause resolved first.

Wiring the Log Into Wazuh

With a live log file in place, I added a new `<localfile>` block to the Wazuh agent's configuration (`ossec.conf`) on `ubuntu-victim`. Because the forwarded file mixes plain `nginx` access log lines with multi line ModSecurity audit blocks, not clean JSON, I set the log format to `syslog` rather than `json`, which handles arbitrary text generically.

Two issues surfaced during this step and were resolved in place:

- `ossec.conf` contains multiple `<ossec_config>` sections; my initial insert command matched every closing tag and duplicated the new block. I caught this by grepping line numbers before restarting the agent, and removed the duplicate with a targeted line range deletion.
- After the fix, the agent restarted cleanly with no configuration errors, and its internal log confirmed the logcollector was actively analyzing the new file.

```
ubuntu-victim@ubuntu-victim:~$ sudo systemctl restart wazuh-agent
ubuntu-victim@ubuntu-victim:~$ sudo systemctl status wazuh-agent --no-pager
● wazuh-agent.service - Wazuh agent
   Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: enabled)
   Drop-In: /etc/systemd/system/wazuh-agent.service.d
            └─override.conf
   Active: active (running) since Thu 2026-09-10 17:59:29 UTC; 57s ago
 Invocation: 76e40e4df34343da8a16c7605c2ab910
   Process: 11365 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 29 (limit: 1504)
   Memory: 220.8M (peak: 261.9M)
      CPU: 1min 3.600s
   CGroup: /system.slice/wazuh-agent.service
           └─11387 /var/ossec/bin/wazuh-execd
             └─11420 /var/ossec/bin/wazuh-agentd
               └─11435 /var/ossec/bin/wazuh-syscheckd
                 └─11445 /var/ossec/bin/wazuh-logcollector
                   └─11459 /var/ossec/bin/wazuh-modulesd

Sep 10 17:59:21 ubuntu-victim systemd[1]: Starting wazuh-agent.service - Wazuh agent...
Sep 10 17:59:22 ubuntu-victim env[11365]: Starting Wazuh v4.14.1...
Sep 10 17:59:24 ubuntu-victim env[11365]: Started wazuh-execd...
Sep 10 17:59:25 ubuntu-victim env[11365]: Started wazuh-agentd...
Sep 10 17:59:26 ubuntu-victim env[11365]: Started wazuh-syscheckd...
Sep 10 17:59:26 ubuntu-victim env[11365]: Started wazuh-logcollector...
Sep 10 17:59:27 ubuntu-victim env[11365]: Started wazuh-modulesd...
Sep 10 17:59:29 ubuntu-victim env[11365]: Completed.
Sep 10 17:59:29 ubuntu-victim systemd[1]: Started wazuh-agent.service - Wazuh agent.
ubuntu-victim@ubuntu-victim:~$
```

Wazuh agent restarting cleanly after the new `<localfile>` block was added, with logcollector and all other components starting without error.

Discovery: No Custom Decoder Needed

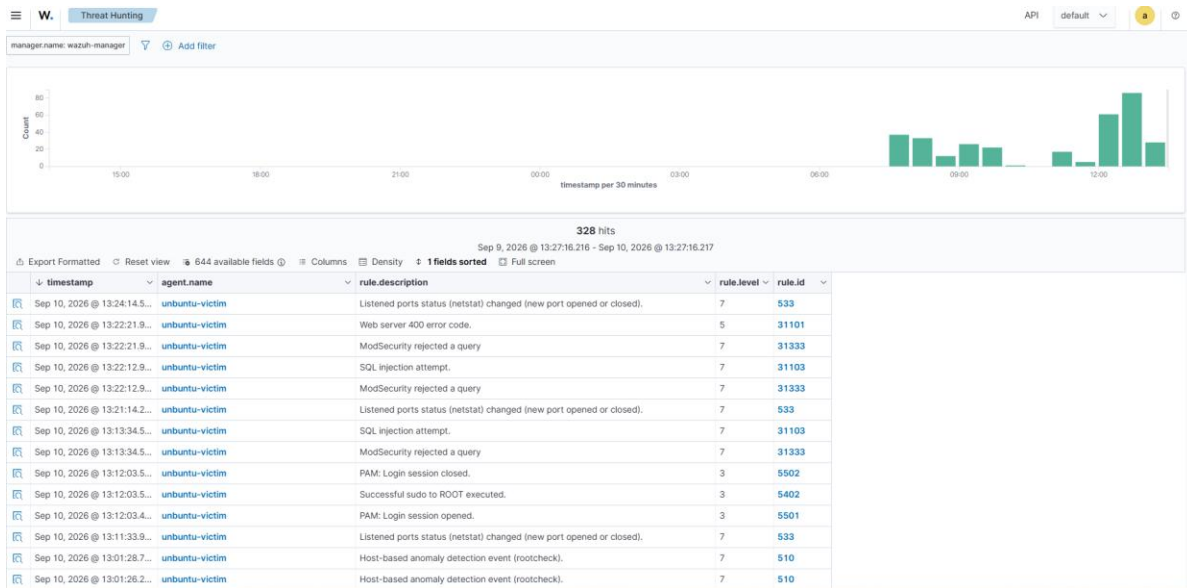
My expectation going in was that Wazuh's default ruleset would not recognize this custom log format, requiring a hand written decoder. Testing showed otherwise: Wazuh's built in web accesslog decoder correctly parsed the forwarded lines, extracting fields such as source IP and requested URL, and its default ruleset already includes rules that classify this exact traffic pattern: rule 31103 ("SQL injection attempt") and rule 31333 ("ModSecurity rejected a query"), both firing together on the same request. This was a case where confirming the actual behavior before building a workaround avoided unnecessary engineering effort.

Validation: Real Cross VM Attacks

With the pipeline wired up, I validated detection using real attack traffic launched from the Kali VM through a browser, against DVWA's SQL Injection module (Security Level: Low), routed through the WAF's port (8081) rather than DVWA's direct port:

- Payload 1: `%' UNION SELECT user, password FROM users-- -`, blocked with HTTP 403
- Payload 2: `1' OR '1'='1`, blocked with HTTP 403

I confirmed both attacks in the Wazuh dashboard as paired, correlated alerts, rule 31103 immediately followed by rule 31333 for each request, rather than a single generic log entry:



Two attacks, each producing a paired detection: rule 31103 (SQL injection attempt) and rule 31333 (ModSecurity rejected a query).

I drilled into a single alert's full document and confirmed the attack was correctly attributed to the real attacker machine, not a loopback or synthetic test:

Document Details		View surrounding documents	View single document
Table JSON			
index	wazuh-alerts-4.x-2026.09.10		
agent.id	001		
agent.ip	10.0.2.3		
agent.name	unbuntu-victim		
data.id	403		
data.protocol	GET		
data.srcip	10.0.2.15		
data.url	/vulnerabilities/sql/?id=%25%27+UNION+SELECT+user%2C+password+FROM+users--+&Submit=Submit		
decoder.name	web-accesslog		
full_log	10.0.2.15 - - [10/Sep/2026:18:21:48 +0000] "GET /vulnerabilities/sql/?id=%25%27+UNION+SELECT+user%2C+password+FROM+users--+&Submit=Submit HTTP/1.1" 403 146 "http://10.0.2.3:8081/vulnerabilities/sql/" "Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0"		
id	1789064532.546318		
input.type	log		
location	/var/log/modsec-waf.log		
manager.name	wazuh-manager		
rule.description	SQL injection attempt.		
rule.firedtimes	2		
rule.gdpr	IV_35.7.d		
rule.groups	web, accesslog, attack, sql_injection		
rule.id	31103		
rule.level	7		
rule.mail	false		
rule.mitre.id	T1190		

Full alert detail: data.srcip (10.0.2.15) matches the Kali attacker VM, with the complete raw request, payload, response code, and user agent captured in full_log.

MITRE ATT&CK and Compliance Mapping

Rule 31103 carries automatic framework mappings, giving this detection immediate relevance to enterprise SOC reporting:

Framework	Mapping
MITRE Technique	T1190, Exploit Public Facing Application
MITRE Tactic	Initial Access
PCI DSS	6.5, 11.4, 6.5.1
GDPR	IV_35.7.d
NIST 800-53	SA-11, SI-4

Results Summary

- Deployed a containerized ModSecurity WAF (OWASP CRS) in front of DVWA, proxying and inspecting traffic on a dedicated port
- Diagnosed a broken native audit log output and built a systemd based workaround to persist WAF logs to disk
- Integrated the forwarded log into Wazuh via a correctly formatted <localfile> block, resolving a multi section config file gotcha along the way
- Confirmed Wazuh's default ruleset already classifies this traffic without custom decoder work, validated rather than assumed
- Ran real attacks from a separate attacker VM and confirmed end to end detection with correct source IP attribution, full request forensics, and automatic MITRE ATT&CK / compliance mapping

Skills Demonstrated

- Web Application Firewall deployment and reverse proxy configuration (ModSecurity, OWASP CRS, nginx)
- Log pipeline engineering: diagnosing a broken logging path and building a systemd based forwarding workaround
- SIEM agent configuration (Wazuh ossec.conf, localfile blocks, log format selection)
- XML configuration debugging in a file with multiple top-level sections
- SIEM correlation rule analysis and validation methodology (testing assumptions before building custom decoders)
- MITRE ATT&CK and compliance framework mapping
- End to end attack execution and detection validation across a multi VM lab